FAIRLANE · PHASE II REHEARSAL GUIDE · FILE 1 OF 3

NFS Shared Storage

Start the shared storage and add new users

Team Orchestrators · Fairlane: A Fair and Fault-Tolerant Scheduler for Distributed ML Training Workloads

WHO IS WHO

- NFS SERVER = Vansh's PC (the shared folder lives here: /srv/fairlane)
- NFS CLIENT = suryansh-vm (sees the same folder at: /mnt/shared_jobs)

WHAT NFS DOES IN FAIRLANE

one line to say to the evaluator

"One folder on the server looks like a normal local folder on every machine, so workers read users' scripts and write logs without us writing any file-transfer code."

FIXED ACCOUNT NUMBERS

must be the same on every machine

`fl_worker / compute_workers`	**2100 (the account jobs run as)**
`fl_vansh`	**2001**
`fl_suryansh`	**2002**
`next free number`	**2003**

HOW TO READ THIS FILE

- Lines under `>> PASTE:` are commands. Copy them exactly, one line at a time, and press Enter after each.
- Anything in **<ANGLE_BRACKETS>** must be replaced with the real value (and remove the < >).
- `EXPECT:` tells you what a correct result looks like.

PART 0 Before You Start (Both PCs, 2 Minutes)

0.1 Same network

Connect BOTH laptops (Vansh's PC and Suryansh's Windows PC) to the SAME phone hotspot. Do NOT use college Wi-Fi: it blocks laptop-to-laptop traffic.

0.2 Suryansh: VM network must be `Bridged`

VirtualBox → select the VM → Settings → Network → `Attached to: Bridged Adapter`, Name = your Wi-Fi adapter. Then, inside the VM, refresh the network:

PASTE suryansh-vm

```
sudo nmcli networking off && sudo nmcli networking on
```

0.3 Write down both IPs

PASTE Both machines

```
hostname -I
```

Take the FIRST address only (ignore 172.17.x.x, that is Docker).

VANSH_IP _____

VM_IP _____

EXPECT
the first three numbers look alike (e.g. 10.190.94.12 and 10.190.94.57). If they are very different, you are not on the same network - go back to Step 0.1.

0.4 Work out the network address (SUBNET)

Take VANSH_IP, replace the last number with 0, and add /24. Example: VANSH_IP 10.190.94.174 → SUBNET 10.190.94.0/24

SUBNET _____

0.5 The nfs scripts must be on both machines

The helper scripts live in the repo folder `nfs/` (5 files - registry.conf, sync_accounts.sh, server_dirs.sh, join.sh, verify.sh).

Vansh: check they are in your repo:

PASTE Vansh's PC

```
cd ~/Code/GPU_Orchestrator
ls nfs
```

Suryansh: fetch only that folder from the main branch into your clone:

PASTE suryansh-vm

```
cd ~/Distributed_GPU_Workload_Orchestrator
git fetch origin main
git checkout origin/main -- nfs
ls nfs
```

EXPECT (both)
join.sh registry.conf server_dirs.sh sync_accounts.sh verify.sh

If `ls nfs` fails on the VM, Vansh has not pushed nfs/ to main yet. Vansh pushes it, then Suryansh repeats this step.

PART 1 Start the NFS Server (Vansh's PC)

1.1 Make sure the NFS server is running

PASTE Vansh's PC

```
sudo systemctl enable --now nfs-server
systemctl status nfs-server --no-pager
```

> **EXPECT**
> `Active: active (exited)`. `exited` is normal for NFS.

1.2 Allow the hotspot network to use the share

The hotspot gives new IPs every time, so this line must match today's SUBNET.

PASTE Vansh's PC

```
sudo nano /etc/exports
```

Make sure the file contains exactly ONE fairlane line like this (replace **<SUBNET>** with the value from Step 0.4; delete any older fairlane lines):

```
/srv/fairlane 127.0.0.1(rw,sync,no_subtree_check) <SUBNET>(rw,sync,no_subtree_check)
```

Save: Ctrl+O, Enter. Exit: Ctrl+X.

1.3 Apply the change

PASTE Vansh's PC

```
sudo exportfs -ra
sudo exportfs -v
```

> **EXPECT**
> /srv/fairlane listed twice (127.0.0.1 and your SUBNET), with `rw` and `root_squash` in the brackets. No output from -ra = success.

1.4 Open the firewall for NFS (only matters if ufw is active)

PASTE Vansh's PC

```
sudo ufw status
```

If it says `Status: active`, run:

PASTE Vansh's PC

```
sudo ufw allow from <SUBNET> to any port 2049 proto tcp
```

If it says `inactive`, skip this step.

1.5 Create/repair accounts and user folders

PASTE Vansh's PC

```
cd ~/Code/GPU_Orchestrator
sudo bash nfs/server_dirs.sh
```

EXPECT

`OK: accounts on ... match the registry`, `folders ready for fl_vansh`, `folders ready for fl_suryansh`, then a listing of users/ and logs/.

1.6 Server checks itself (mounts its own share and tests permissions)

PASTE Vansh's PC

```
sudo bash nfs/join.sh 127.0.0.1
```

EXPECT

last line `ALL CHECKS PASSED`.

PART 2 Connect the Client (suryansh-vm)

2.1 Check the VM can reach Vansh's PC

PASTE suryansh-vm

```
ping -c 3 <VANSH_IP>
```

EXPECT

`3 received`. If 0 received → Part 0 again (hotspot / Bridged).

2.2 Join the share (installs the NFS client, creates the same accounts with the same numbers, mounts, and runs the checks)

PASTE suryansh-vm

```
cd ~/Distributed_GPU_Workload_Orchestrator
sudo bash nfs/join.sh <VANSH_IP>
```

EXPECT

the four sections `== 1 ... == 4` and finally:

```
PASS  share mounted at /mnt/shared_jobs
PASS  worker can read users/fl_vansh
PASS  worker can write logs/fl_vansh
PASS  worker cannot write users/fl_vansh
PASS  fl_suryansh cannot read fl_vansh's folder
ALL CHECKS PASSED
```

2.3 Live proof: one file, two machines

PASTE Vansh's PC

```
echo "hello from Vansh's PC" | sudo -u fl_vansh tee /srv/fairlane/users/fl_vansh/hello.txt
```

PASTE suryansh-vm

```
sudo -u fl_worker cat /mnt/shared_jobs/users/fl_vansh/hello.txt
```

EXPECT on the VM

hello from Vansh's PC

SAY TO THE EVALUATOR

"Vansh wrote the file on his PC; the worker on the other machine reads it instantly through NFS. No copying."

2.4 Live proof: users are isolated

PASTE suryansh-vm

```
sudo -u fl_suryansh ls /mnt/shared_jobs/users/fl_vansh
```

EXPECT

`Permission denied`

SAY TO THE EVALUATOR

"Each user folder is mode 750, owned by the user with group compute_workers. The worker can read it, other students cannot."

PART 3 Add a New User (Example: fl_harsh, Number 2003)

RULE numbers are assigned ONLY in nfs/registry.conf, and must be the same on every machine. Never reuse a number.

3.1 Vansh: add the user to the registry

PASTE Vansh's PC

```
cd ~/Code/GPU_Orchestrator
nano nfs/registry.conf
```

Change these two lines:

```
# User IDs: 2001-2099. Next free: 2003. Never reuse a number.
USERS="fl_vansh:2001 fl_suryansh:2002"
```

to:

```
# User IDs: 2001-2099. Next free: 2004. Never reuse a number.
USERS="fl_vansh:2001 fl_suryansh:2002 fl_harsh:2003"
```

Save: Ctrl+O, Enter. Exit: Ctrl+X.

3.2 Vansh: create the account and folders on the server

PASTE Vansh's PC

```
sudo bash nfs/server_dirs.sh
```

EXPECT

`created group fl_harsh (2003), created user fl_harsh (2003), folders ready for fl_harsh.`

3.3 Suryansh: create the same account on the VM

(For the rehearsal, make the same edit to nfs/registry.conf on the VM as in Step 3.1. In real use, Vansh commits the file and everyone runs `git pull`.)

PASTE suryansh-vm

```
cd ~/Distributed_GPU_Workload_Orchestrator
nano nfs/registry.conf
```

(make the same change as Step 3.1, save and exit)

PASTE suryansh-vm

```
sudo bash nfs/sync_accounts.sh
```

EXPECT

`created user fl_harsh (2003)` and `OK: accounts on Ubuntu match the registry`

3.4 Prove the new user works and is isolated

PASTE suryansh-vm

```
id fl_harsh
ls -ln /mnt/shared_jobs/users
sudo -u fl_harsh ls /mnt/shared_jobs/users/fl_vansh
```

EXPECT

```
uid=2003(fl_harsh) gid=2003(fl_harsh) ...
a folder owned by 2003 in the listing
"Permission denied" for the last command
```

SAY TO THE EVALUATOR

"Adding a user is one line in the registry and one script per machine. Because the number is fixed, the files belong to the same person on every machine."

3.5 (After the rehearsal) undo the test user, if you want

Remove `:fl_harsh:2003` from USERS in registry.conf on both machines. The leftover account does no harm; you can leave it.

If Something Goes Wrong

PROBLEM	FIX
ping fails	Not on same hotspot, or VM not Bridged. Redo Part 0.
`access denied by server while mounting`	/etc/exports SUBNET is wrong or not applied. Redo Steps 1.2-1.3.
Mount hangs / times out	Firewall on Vansh's PC. Step 1.4.
`ERROR: UID 2003 is already used by ...`	That number is taken on this machine. Pick the next free number in the registry and use it on ALL machines.
`already mounted` but checks fail	Remount: sudo umount /mnt/shared_jobs then rerun join.sh.
`Run with sudo`	Put `sudo` in front of the command.

Known Limitation (if the evaluator asks)

Plain NFS trusts the user number a client sends. Someone with root on their own laptop could pretend to be another user. Fixing this needs Kerberos (NFSv4 + krb5), which is out of scope; we assume a trusted lab network. root_squash is on, so `sudo` on a client does NOT give root over the shared files.